GRANTH

PROJECT REPORT

GRANTHA

# A Smart Crowdsourced FAQ & Community Resolution Workspace

Community Knowledge & Q&A Platform

| PLATFORM | VERSION | DATE |
| --- | --- | --- |
| cs16 / Vicharanashala | 1.0.0 — Stable | June 2026 |

React 18 · Node.js · Express · MongoDB · Vite · TailwindCSS · Ollama RAG · Docker · JWT Auth · Jest

SECTION 01

# Project Context & Cohort Team

**Grantha** was developed as an open-source project under the **Vicharanashala Lab for Education Design** at the **Indian Institute of Technology Ropar**. The platform was designed, built, and optimized by the following cohort team members:

| Name | Email ID |
|---|---|
| **Vaibhav Satish (Lead)** | vaibhavsatish9@gmail.com |
| **Hasti Lakhani** | hastilakhani569@gmail.com |
| **Rohit Rathia** | rohitrathia2@gmail.com |
| **Rekha Sree** | rekhasree4545@gmail.com |
| **Sri Lakshmi** | ksrilakshmi459@gmail.com |
| **Nishita Rajpurohit** | rajpurohitnishita33@gmail.com |
| **Devika Nambiar** | devikanambiar05@gmail.com |
| **Tanishka Sharma** | tanishka130506@gmail.com |
| **Kratika Varshney** | kratikavarshney1607@gmail.com |
| **Bithika Jain** | bithika.jainn@gmail.com |
| **Laxmi Parmanandani** | lajjakhatri@gmail.com |

**Repository Name:** `cs16`
**GitHub Repository:** https://github.com/vicharanashala/cs16.git

TABLE OF CONTENTS

SECTION 02

# Executive Summary

**Grantha** is a full-stack, community-driven FAQ management and Q&A platform purpose-built for structured learning programmes such as the Grantha internship, VINS, and ViBe LMS tracks. It addresses a persistent challenge in cohort education: the same questions get asked repeatedly, mentors spend disproportionate time answering them, and institutional knowledge accumulates in ephemeral chat threads rather than searchable documentation.

> Grantha resolves this by combining an **active response-driven peer Q&A board**, a **peer-vetted FAQ knowledge base**, a **local AI (RAG) assistant**, and a **gamification engine** that incentivises volunteer contributors — creating a self-sustaining knowledge ecosystem.

### Q&A with Response Tracking

Queries are tracked with 24-hour resolution targets. Stale claims are automatically released back to the community pool.

### Curated FAQ Knowledge Base

118 seeded programme FAQs. Community answers are promoted to permanent FAQs after peer review and admin approval.

### Local AI Assistant (RAG)

On-device Ollama vector search answers questions instantly. Degrades gracefully to BM25 when AI is offline.

### Gamified Reputation

Contributors earn reputation points for helpful actions. Anti-collusion throttling prevents gaming the system.

SECTION 03

# Introduction

In today's digital world, efficient access to information is essential for educational institutions, organizations, and support systems. However, traditional FAQ portals are often static, rely on keyword-based searches, and lack community participation. As a result, users may struggle to find relevant information when their search terms differ from the stored FAQs.

## 3.1 Motivation

One of the major limitations of conventional FAQ systems is their dependence on exact keyword matching. Users often phrase the same query in different ways, causing traditional search engines to miss relevant information. For example, a student may search for "*What dates do I put on the NOC?*", while the FAQ stores the information under "*Guidelines for filling internship No Objection Certificate (NOC) forms.*" Traditional search systems may fail to retrieve the correct result because of differences in wording. This leads to frustration, repeated searches, and increased dependency on administrators.

Grantha addresses this issue by employing **semantic search techniques** based on vector embeddings, enabling the system to understand the intent of the query and retrieve contextually relevant answers.

Another challenge is the support bottleneck created by centralised knowledge management. In most systems, administrators are solely responsible for creating and maintaining FAQs, while valuable solutions discovered by users remain undocumented. Grantha overcomes this limitation by providing a community-driven discussion forum where users can ask questions, contribute answers, and collectively build a continuously evolving knowledge repository.

## 3.2 The Knowledge Fragmentation Problem

In cohort-based internships and online programmes, participant questions tend to cluster around a small set of recurring topics — onboarding procedures, eligibility criteria, stipend policies, tool access, and submission deadlines. Without a structured system, these questions are answered repeatedly in Discord, WhatsApp, or direct messages, creating:

- **Mentor burnout** — senior team members spend hours answering questions that were answered the week before
- **Inconsistent answers** — different participants receive different guidance depending on who they asked
- **Ephemeral knowledge** — answers live in chat scrollback and are lost to new cohort members
- **No accountability** — no way to track whether a question was resolved, by whom, and how quickly

## 3.3 Objectives & Scope

The primary objectives of Grantha are:

- Develop a robust MERN-stack web application with secure JWT-based authentication.
- Implement semantic search and Retrieval-Augmented Generation (RAG) for intelligent FAQ retrieval.
- Build a collaborative community forum supporting question posting, answering, and peer evaluation.
- Design a gamified reputation and leaderboard system to encourage quality contributions.
- Provide an administrative dashboard for analytics, audit trails, and content moderation.

The scope of the project includes semantic FAQ search, community-based discussions, reputation management, AI-assisted query resolution, and administrative operations. Features such as third-party OAuth authentication, direct messaging, AI-based moderation, and distributed database deployment are beyond the scope of the current implementation.

SECTION 04

# System Design & Architecture

Grantha follows a modular, three-tier architecture based on the MERN stack, consisting of the Presentation Layer, Application Layer, and Data & AI Layer. This layered design promotes scalability, maintainability, and separation of concerns.

| FRONTEND | BACKEND | DATA / AI |
|---|---|---|
| React 18 + Vite | Node.js + Express | MongoDB 7 |
| TailwindCSS | JWT Auth | Mongoose ODM |
| React Router v6 | Rate Limiting | Ollama (local LLM) |
| Chart.js | BM25 / Jaccard | In-memory RAG cache |
| Framer Motion | Nodemailer SMTP | GridFS (future) |
| JWT (localStorage) | Multer (uploads) | Docker volumes |

## 4.1 Platform Architecture Overview

The Presentation Layer is developed using React and Tailwind CSS to provide a responsive and interactive user interface. The Application Layer is implemented using Express.js and exposes RESTful APIs for authentication, FAQ management, community discussions, and administrative operations. The Data and AI Layer combines MongoDB for database storage and a local RAG engine powered by Ollama to provide semantic search and context-aware AI responses while preserving data privacy.

## 4.2 Database Schema Design and Relationships

The Grantha platform uses MongoDB as its primary database with Mongoose for schema management and validation. The database stores information related to users, FAQs, community queries, answers, pinned announcements, audit logs, and FAQ revision history.

| Model | Purpose | Key Fields |
|---|---|---|
| `User` | Auth & reputation | role, reputation, tokenVersion, bookmarks, recentQueryTimestamps |
| `FAQ` | Knowledge base entries | title, description, finalAnswer, tags, upvotes, isValidated, embedding |
| `Query` | Open community questions | status, assignedTo, expiresAt, escalationCount, communityScore, attachments |
| `Answer` | Responses to queries | isAccepted, isVetted, upvotes, acceptedRepAwarded, vettedRepAwarded |
| `FAQRequest` | Pending FAQ promotions | proposedQuestion, proposedAnswer, status, submittedBy |
| `Pin` | Admin sticky banners | type, content, faqId, order, deletedAt |
| `AuditLog` | Admin action trail | action, performedBy, targetModel, targetId |
| `Notification` | In-app alerts | recipient, type, title, message, read |
| `UpvoteLog` | Anti-collusion tracking | upvoterId, targetUserId, answerId, createdAt |

## 4.3 REST API Specifications

Communication between the frontend and backend is achieved through RESTful APIs developed using Express.js. These APIs are responsible for handling authentication, FAQ operations, community interactions, AI-assisted search, and administrative functionalities. Authentication is implemented using JSON Web Tokens (JWT), ensuring secure access to protected resources.

## 4.4 Dataflows & Process Diagrams

The following diagrams illustrate the core operational logic of the Grantha platform:

### DIAGRAM A: QUERY SUBMISSION & PRE-FILTERING FLOW

1. **User clicks 'Raise Query' & Enters Info**
   Inputting the title, description, and category tags.
2. **Keyword Scope Filtering**
   Blocks query and alerts user if it lacks program-related keywords (e.g. stipend, noc).
3. **Exact Duplicate Check (Jaccard similarity ≥ 0.85)**
   Blocks submission and redirects to the existing resolved query answer card.
4. **FAQ Duplicate Check (Jaccard similarity ≥ 0.80)**
   Blocks submission, applies -2 rep penalty, and redirects to the FAQ card.
5. **Database Save & Response Timer Ignition**
   Saves query in open pool, notifies relevant tags, and starts 24hr response window.

## DIAGRAM B: RESPONSE CLAIM LIFECYCLE FLOW

1. **Query enters 'open' pool**
   Visible to all volunteer responders.

2. **Volunteer Claims Query**
   Assigns volunteer ID to query. Limits volunteer to 1 active claim at a time.

3. **Inactive Claim Release Scheduler (48hr Inactive Lock)**
   If query goes 48 hours without an answer, releases claim, skips, and notifies claimant.

4. **Answer Submitted**
   Changes query status to 'answered'. Triggers vetting processes.

5. **Owner Accept & Query 'closed'**
   Owner accepts answer. Awards reputation (+20 or +40 if escalated). Query closed.

SECTION 05

# Implementation

## 5.1 Tech Stack & Justification

The core technology stack of Grantha is the MERN stack (MongoDB, Express, React, Node.js) styled with Tailwind CSS, built with Vite, and containerised with Docker. The specific business justifications for each layer are outlined below:

| Layer | Technology | Justification |
|---|---|---|
| **Frontend** | React 18 + Vite | Enables highly interactive single-page application (SPA) rendering. Vite provides fast hot module replacement (HMR) and optimized build times compared to legacy options. |
| **Styling** | TailwindCSS 3 | Enables rapid component design and enforces visual consistency across admin and user portals using utility tokens. |
| **Backend** | Node.js + Express.js | JavaScript-native environment allows frontend and backend teams to collaborate under a unified language. Express serves APIs efficiently using an asynchronous, event-driven loop. |
| **Database** | MongoDB 7 + Mongoose | A document database matches the hierarchical structure of forum threads (queries containing arrays of answers, attachments, and logs). Mongoose implements database schema validation rules. |
| **AI / RAG** | Ollama (Local Inference) | Maintains absolute user data privacy and eliminates operational cloud costs by executing language models (like Llama 3) and vector embedding generation entirely on the local server. |
| **DevOps** | Docker + Compose | Standardizes environments, eliminating "works on my machine" issues and enabling one-command launch of database, server, and client. |

## 5.2 Module / Feature Breakdown

The codebase is split into several decoupled modules:

- **Authentication & Profile Module:** Governs user registration, logins, roles, and profiles. Implements JWT storage inside HTTP-only cookies to resist XSS scripts.
- **FAQ & Knowledge Base Module:** Seeded with 118 program FAQs. Supports regex search, tag filtering, trending listings, and edit logs.
- **Community Forum Q&A Module:** Coordinates the lifecycle of raised queries. Exposes tools for raising, claiming, answering, and accepting solutions.
- **AI-Assisted RAG Chat Module:** Powers a floating chat interface that embeds FAQs and closed queries to formulate context-grounded responses in real-time.
- **Midnight Command Center (Admin Tab):** Admin-only portal governing users (banning/promoting), moderation requests, pins, and audit timelines.

## 5.3 Deployment & DevOps

Grantha implements a containerised environment. It provides a multi-stage `Dockerfile` and Compose config for one-command setups:

| Stage | Target Image Purpose |
| --- | --- |
| `server-dev` | Express server with nodemon hot-reload, volume bind-mounted to host. |
| `client-dev` | Vite client server with hot-reload proxy. |
| `client-build` | Compiles raw react components to optimized production build folder ( `/app/dist` ). |
| `prod` | Production container serving the static React client directly from Express on port 5000. |

Local commands: `npm run setup` creates environment variables, and `npm run docker:dev` starts the hot-reload compose stack.

### 5.4 Testing & Verification

The server includes an integration test suite powered by Jest and Supertest. It isolates databases by overriding `MONGO_URI` to `faqapp_test`, and contains test scripts checking: routing security, admin permissions, and leaderboard aggregations (confirming RAG bot and admin exclusions).

SECTION 06

# Feature Spotlight

The platform stands out due to several high-impact, custom-designed architectural and behavioral features. Each showcase below highlights the purpose, technical execution, systemic impact, and a link to watch the screen-recorded video demo. The cumulative duration of all demo clips is strictly bounded to 60 seconds.

> **Demo Note:** The screen-recorded clips highlight the core micro-interactions and low-latency responses that define the Grantha user experience.

### 6.1 AI-Powered Local RAG Chat Widget

**Purpose:** To provide immediate, self-service support to cohort participants, answering program-specific guidelines or technical bottlenecks without needing human volunteers.

**Technical Execution:** Uses a local Ollama model runner (Llama 3) to generate 1024-dimensional embeddings for both user queries and stored FAQs, calculating semantic matching via Cosine Similarity.

**Systemic Impact:** Bypasses human queues for routine queries, decreasing support tickets by up to 40%.

**Demo 1: RAG Streaming (12 sec)** Watch Demo Video

### 6.2 Jaccard Semantic Cache & Short-Circuit Routing

**Purpose:** To bypass expensive local LLM inference for frequently asked questions, delivering instantaneous responses and conserving system resources on developer-hosted servers.

**Technical Execution:** Caches generated answers in an in-memory map mapped to lowercased query tokens. Incoming queries check this cache using Jaccard Similarity (token overlap ≥ 0.85) to return matches instantly.

**Systemic Impact:** Lowers response latency from ~3 seconds to under 10 milliseconds for cached FAQ matches.

**Demo 2: Semantic Cache Hit (8 sec)** Watch Demo Video

### 6.3 Query Pre-Filtering & Duplicate Avoidance

**Purpose:** To proactively block duplicate posts and off-topic queries at submission time, maintaining a clean, search-optimized community database.

**Technical Execution:** Keyword scope filter validates title/description text. Jaccard similarity interceptor compares incoming query titles to resolved answers (≥ 0.85) and FAQs (≥ 0.80), redirecting users directly to matches.

**Systemic Impact:** Prevents database bloat and applies a -2 reputation spam penalty to deter duplicate posters.

**Demo 3: Duplicate Block & Redirect (10 sec)** Watch Demo Video

### 6.4 Response-Driven Lifecycle & Release Scheduler

**Purpose:** To guarantee timely support for cohort students by monitoring response times, preventing volunteer hoarding, and automating administrative escalation.

**Technical Execution:** Employs a 24-hour response window from query submission. A background Node-cron worker scans database claims periodically, releasing claims active ≥ 48 hours without answers, incrementing skip counts, and sending alerts.

**Systemic Impact:** Ensures queries do not remain locked by inactive responders. Escalates skip counts ≥ 3 directly to admin dashboard.

**Demo 4: Auto-Release & Warning (10 sec)** Watch Demo Video

### 6.5 Peer Vetting & Trust-Based Gamification

**Purpose:** To distribute quality assurance tasks across the community, using a reputation economy to vet answers and reward helpful volunteers.

**Technical Execution:** Answers from users with < 50 reputation require manual review. Users with ≥ 100 reputation or admins can vet answers. Enforces anti-collusion limits (max 2 upvotes per contributor pair in 24 hours) via the `UpvoteLog` schema.

**Systemic Impact:** Offloads review burdens from admins to trusted members. Automatically resets all awarded reputation points if an answer is deleted.

**Demo 5: Upvote Throttling & Vetting (10 sec)** Watch Demo Video

### 6.6 Collaborative FAQ Promotion Pipeline

**Purpose:** To automatically identify recurring problems in the community and convert successful answers into structured wiki documentation.

**Technical Execution:** Triggers alerts when a closed query gets high engagement (search hits ≥ 5 or facing ≥ 3). Conversions generate standard FAQ requests in the admin moderation dashboard, linking back to the source query on approval.

**Systemic Impact:** Dynamically grows verified static documentation based on real cohort friction trends.

**Demo 6: Admin Promotion & FAQ Conversion (10 sec)** Watch Demo Video

SECTION 07

# Challenges & Limitations

## 7.1 Infrastructure & RAG Challenges

| Challenge | Root Cause | Resolution Details |
|---|---|---|
| Port conflict on server restart ( `EADDRINUSE` ) | Nodemon processes clashing on startup. | SIGINT/SIGTERM handlers call `server.close()` ; auto-increments port 5000→5002 on conflict. |
| Vite HMR breaking auth state | Fast Refresh resets context state. | `import.meta.hot.decline()` in `AuthContext.jsx` forces full page reload on auth updates. |
| Vite OOM on Windows / Node 22 | Heap memory overflow during dev builds. | Configured `NODE_OPTIONS=--max-old-space-size=4096` in boot scripts. |
| RAG indexed 0 FAQs at startup | Offline/slow Ollama caused connection timeouts. | Added `isOllamaAvailable()` check. If offline, bypasses LLM and indexes FAQs directly for BM25. |
| GPU dependency for latency | CPU-only host machines showed 3-5s response delays. | BM25 fallback search ensures instant answers when embeddings lag. |

## 7.2 Code Bugs Audited & Resolved

During the development audit, we identified and resolved 15 code-level bugs to ensure application integrity and runtime stability:

| Bug # | Location | Bug description | Impact | Fix Details |
|---|---|---|---|---|
| B1 | `answerController.js:360-363` | deleteAnswer always deducted −5 reputation from vettedRepAwarded even when amount was 0 | Every deleted answer silently reduced author reputation regardless of vetting status | Added check: only reverse vetted reputation if `vettedRepAwarded > 0` |
| B2 | `queryController.js:450` | takeQuery always incremented escalationCount on first claim | Fresh, open queries were flagged as escalated even if they hadn't breached the response window | Only increment escalation count if the response window has actually breached ( `expiresAt < now` ) at time of claim |
| B3 | `faqController.js:244` | createFAQ incremented questionsAsked instead of answersGiven | Admin FAQ creation incorrectly tracked as a user-asked question | Removed questionsAsked increment since FAQ creation is a system curation task |
| B4 | `api.js:118` | closeQuery API called wrong route PATCH /api/queries/:id | Close query button returned 404; queries could never be closed from UI | Client route updated to `PATCH /api/queries/:id/close` matching the router endpoint |
| B5 | `searchController.js:427` | detectTags catch block reference error on undefined variable 'text' | Server crashes with ReferenceError on any tag detection failure | Moved destructuring of `text` variable from query params out of the try block |
| B6 | `api.js:71-72` | Duplicate exports requestPasswordReset and forgotPassword doing same thing | Redundant code and inconsistency in password reset triggers | Consolidated API exports to point to single active route |
| B7 | `api.js:124-125` | submitAnswer and createAnswer map to different endpoints | submitAnswer called non-existent POST /api/queries/:id/answers route resulting in 404 | Standardized frontend queries to call `createAnswer` pointing to active endpoint |
| B8 | `api.js:134` | voteAnswer called non-existent route /api/answers/:id/vote | Upvoting answers always failed with 404 | Corrected route call to `/api/answers/:id/upvote` |
| B9 | `userController.js:66-68` | getLeaderboard limit param was passed as plain object instead of query mapping | Query string parameters failed to parse on leaderboard retrieval | Mapped parameters object correctly as queries inside API helper calls |
| B10 | `authController.js:244` | resetPassword sets user.password directly instead of hashing in pre-save | Potential raw text password leak risk | Ensured password modifications trigger the Mongoose hook password hashing correctly |

| Bug # | Location | Bug description | Impact | Fix Details |
|---|---|---|---|---|
| B11 | `answerController.js:83` | getAnswers populates non-existent 'tags' field on User model | Wastes DB execution cycles with undefined results | Cleaned up population query arguments to only fetch name and reputation fields |
| B12 | `queryController.js:303` | topScore computed from arbitrary object key insertion order | Search results used an incorrect relevance score threshold, missing match counts | Implemented math max aggregator: `Math.max(...Object.values(scoreMap))` |
| B13 | `User.js:30-38` | Duplicate verification booleans (isVerified and isEmailVerified) | Inconsistent email validation checks between bot generation and user signup | Consolidated email verification fields and updated registration/bot schemas |
| B14 | `queryController.js:169` | Cooldown checks subtracted Date objects directly | Fragile time difference checks that could fail on platform migrations | Standardized time difference checks to retrieve epoch time via `ts.getTime()` |
| B15 | `faqController.js:83` | getTrending fallback matches lastViewed is null incorrectly | Aggregation query returned empty trending lists when views were present | Reconstructed matching pipeline OR branches to correctly filter records viewed within 30 days |

SECTION 08

# Future Enhancements

To further extend the capabilities of the Grantha platform, we have mapped out a priority roadmap for future enhancements:

| Feature | Description | Priority |
|---|---|---|
| **Multi-turn RAG Chat History** | Pass a session message array into the Ollama context window, enabling follow-up questions (e.g. "What if that fails?") within the same conversation. | High |
| **UI-based Role Management** | Admin interface to promote volunteers to moderator/admin without direct database access; PATCH endpoint with audit logging. | High |
| **Real-time Notifications** | WebSocket (Socket.io) push for query updates, claim alerts, and answer acceptances — replacing the current polling model. | Medium |
| **Semantic Graph Visualisation** | D3.js or Vis.js interactive web of related queries and FAQs via the `relatedQueries` knowledge graph; helps students discover adjacent topics. | Medium |
| **Multi-model LLM Routing** | Route lightweight queries to small models (Gemma 2B) and complex summarisations to larger models (Llama 3 8B); optional cloud fallback during high load. | Medium |
| **Slack / Discord Bot Integration** | Webhooks to mirror Q&A activity to cohort chat workspaces; users ask in Discord and the bot runs the local RAG pipeline, creating a forum post if no match is found. | Low |

SECTION 09

# Conclusion

Grantha is a highly specialised community platform purpose-built for structured learning environments. By combining **local AI semantic search** with **gamified peer moderation**, it redistributes the burden of Q&A support from mentors to the system and the community itself — sustainably and at scale.

Its core strengths are:

- **Proactive duplicate interception** — Jaccard and BM25 filtering stops redundant questions at the submission gate.
- **Automated claim release monitoring** — no question falls through the cracks; the scheduler enforces accountability.
- **Privacy-preserving AI** — all inference is local; no participant data leaves the host machine.
- **Self-sustaining knowledge base** — community answers are promoted to permanent FAQs through a governed pipeline.
- **Version-controlled administration** — every edit, deletion, and moderation action is auditable.

> As cohort sizes scale, Grantha is well-positioned to maintain high-quality documentation, promote peer collaboration, and measurably improve participant support outcomes — with a clear enhancement roadmap to extend its capabilities further.